# Selenium Test Report

**Application:** CollabDoc - Realtime Collaborative Document Server with AI Integration
**Student:** Nitish Kumar | 24BCE1520
**Date of run:** 16 October 2026

**Result: 13 of 13 test cases passed** in 16.5 s. Median keystroke-to-screen sync latency between two browsers: **67.2 ms** (SRS budget 200 ms).

## Test environment

| Item | Value |
|---|---|
| Framework | Python 3 + pytest + Selenium WebDriver 4.51.0 |
| Browser | Google Chromium 141 (headless) via ChromeDriver 141 |
| Sessions | 3 independent browsers: Alice (Owner), Bob (Editor), Carol (Viewer) |
| Application under test | CollabDoc FastAPI server + JS client, SQLite, in-memory broadcaster |
| Waits | Explicit WebDriverWait (50 ms polling), no fixed sleeps for assertions |
| Reports | JUnit XML, pytest-html, screenshots per test case |

## Test cases and results

| ID | Test case | Time (s) | Status |
|---|---|---|---|
| TC01 | New users can create an account and land in the workspace. | 5.13 | **PASS** |
| TC02 | Signing in with a wrong password shows an error and keeps the user out. | 1.76 | **PASS** |
| TC03 | Owner creates a document and renames it; it appears in the sidebar. | 0.48 | **PASS** |
| TC04 | Owner invites Bob as Editor and Carol as Viewer (RBAC). | 1.56 | **PASS** |
| TC05 | Bob opens the shared document; both users see two presence avatars. | 0.42 | **PASS** |
| TC06 | Alice's typing appears for Bob, then Bob's appears for Alice (OT over WebSocket). | 2.59 | **PASS** |
| TC07 | Alice and Bob type at the same moment; both copies converge to identical text. | 0.43 | **PASS** |
| TC08 | Keystroke-to-screen latency between two browsers stays under the 200 ms SRS budget. | 0.98 | **PASS** |
| TC09 | Carol (Viewer) can open the document but the editor is not editable. | 1.61 | **PASS** |
| TC10 | Authors panel tints blocks by author and the stylometric model trains on both writers. | 0.35 | **PASS** |
| TC11 | A comment added by Alice shows up in Bob's comment count without reloading. | 0.48 | **PASS** |
| TC12 | Version history lists saved versions and opens one for preview. | 0.40 | **PASS** |
| TC13 | No severe JavaScript errors were logged in any browser during the run. | 0.34 | **PASS** |

## Latency measurement (TC08)

| Sample | 1 | 2 | 3 | 4 | 5 | Median |
|---|---|---|---|---|---|---|
| Latency (ms) | 83.8 | 57.4 | 64.3 | 67.2 | 73.6 | 67.2 |

Measured from the moment Alice's browser starts typing a marker to the moment Bob's browser holds it, using each page's high-resolution clock. The figure includes WebDriver keystroke overhead, so the true server round trip is lower.

## Execution log

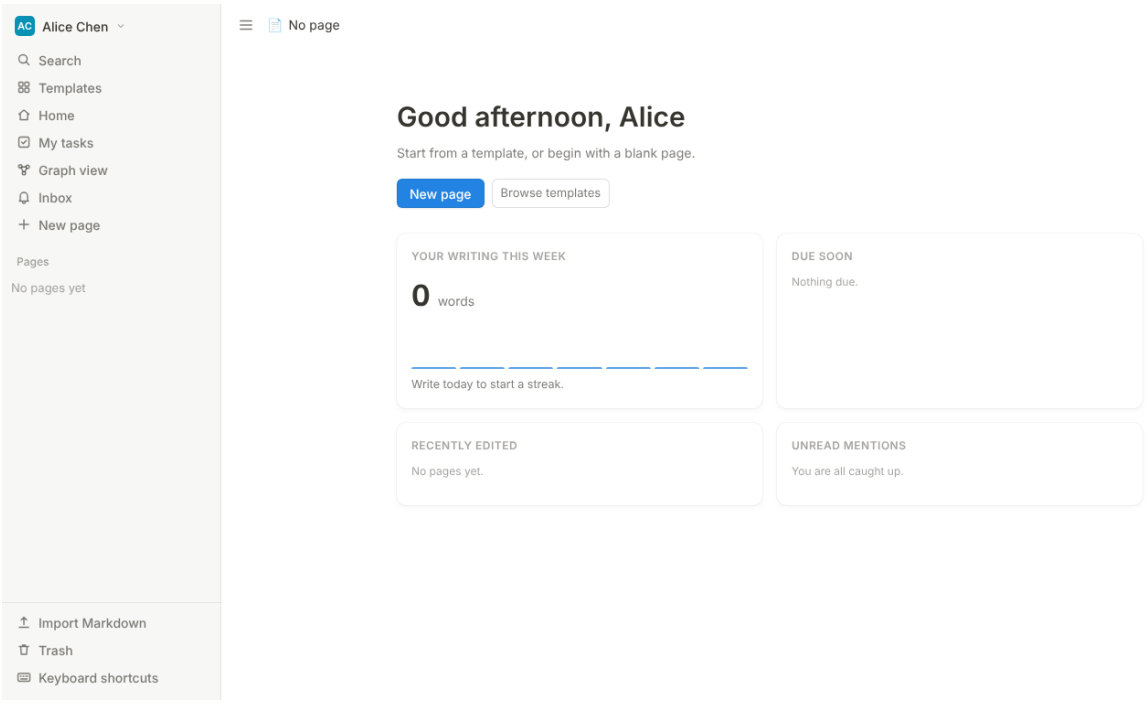
```
============================= test session starts ==============================
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.7.0 -- /usr/bin/python3
collecting ... collected 13 items

test_collabdoc_e2e.py::test_tc01_signup_reaches_workspace PASSED
test_collabdoc_e2e.py::test_tc02_invalid_login_rejected PASSED
test_collabdoc_e2e.py::test_tc03_create_document PASSED
test_collabdoc_e2e.py::test_tc04_share_editor_and_viewer PASSED
test_collabdoc_e2e.py::test_tc05_collaborator_opens_doc_and_presence PASSED
test_collabdoc_e2e.py::test_tc06_realtime_sync_both_directions PASSED
test_collabdoc_e2e.py::test_tc07_concurrent_edits_converge PASSED
test_collabdoc_e2e.py::test_tc08_sync_latency_under_200ms latency samples ms: [83.8, 57.4, 64.3, 67.2, 73.6] median=67.2
PASSED
test_collabdoc_e2e.py::test_tc09_viewer_is_read_only PASSED
test_collabdoc_e2e.py::test_tc10_ai_authorship_attribution PASSED
test_collabdoc_e2e.py::test_tc11_comment_syncs_live PASSED
test_collabdoc_e2e.py::test_tc12_version_history PASSED
test_collabdoc_e2e.py::test_tc13_no_javascript_errors PASSED

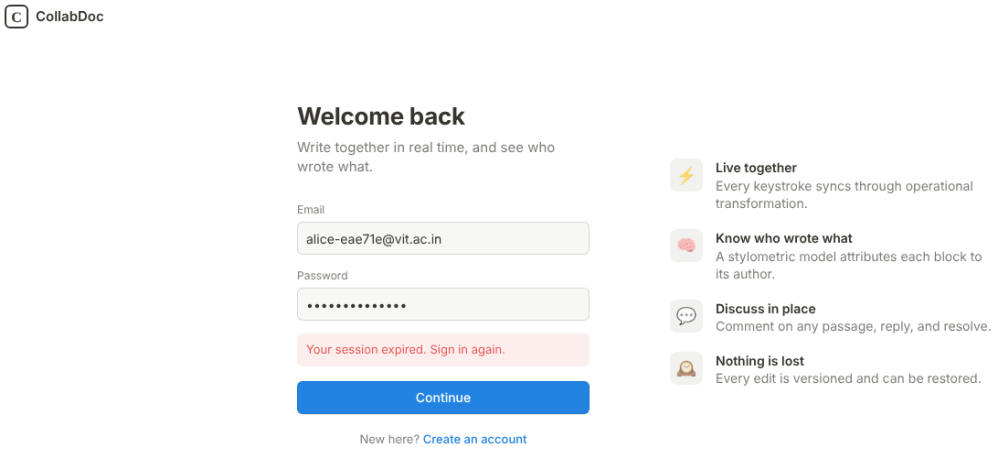
- generated xml file: /tmp/claude-0/-home-claude/cd569568-c387-5e57-96a8-1afff294abf1/scratchpad/seltests/results.xml -
- Generated html report: file:///tmp/claude-0/-home-claude/cd569568-c387-5e57-96a8-1afff294abf1/scratchpad/seltests/selenium_rep
============================== 13 passed in 16.69s ==============================
```

# Screenshots

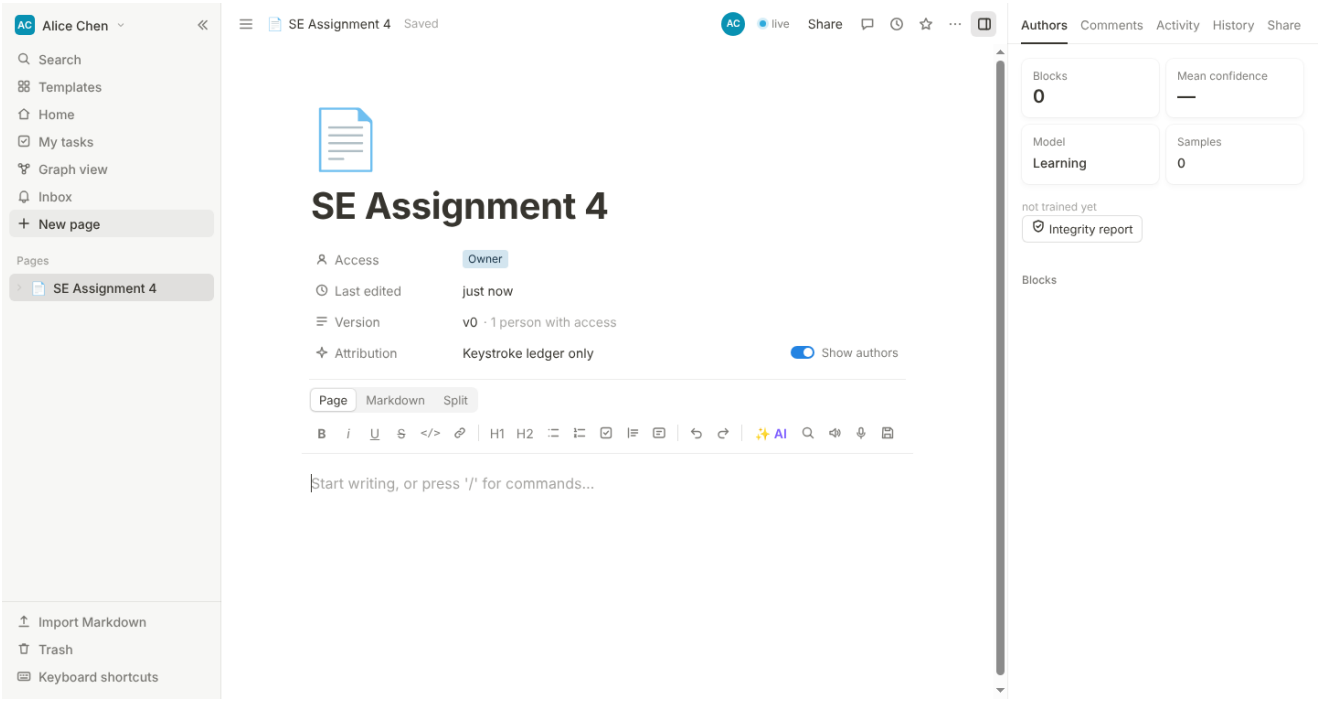
## TC01 - Alice reaches the workspace after sign-up



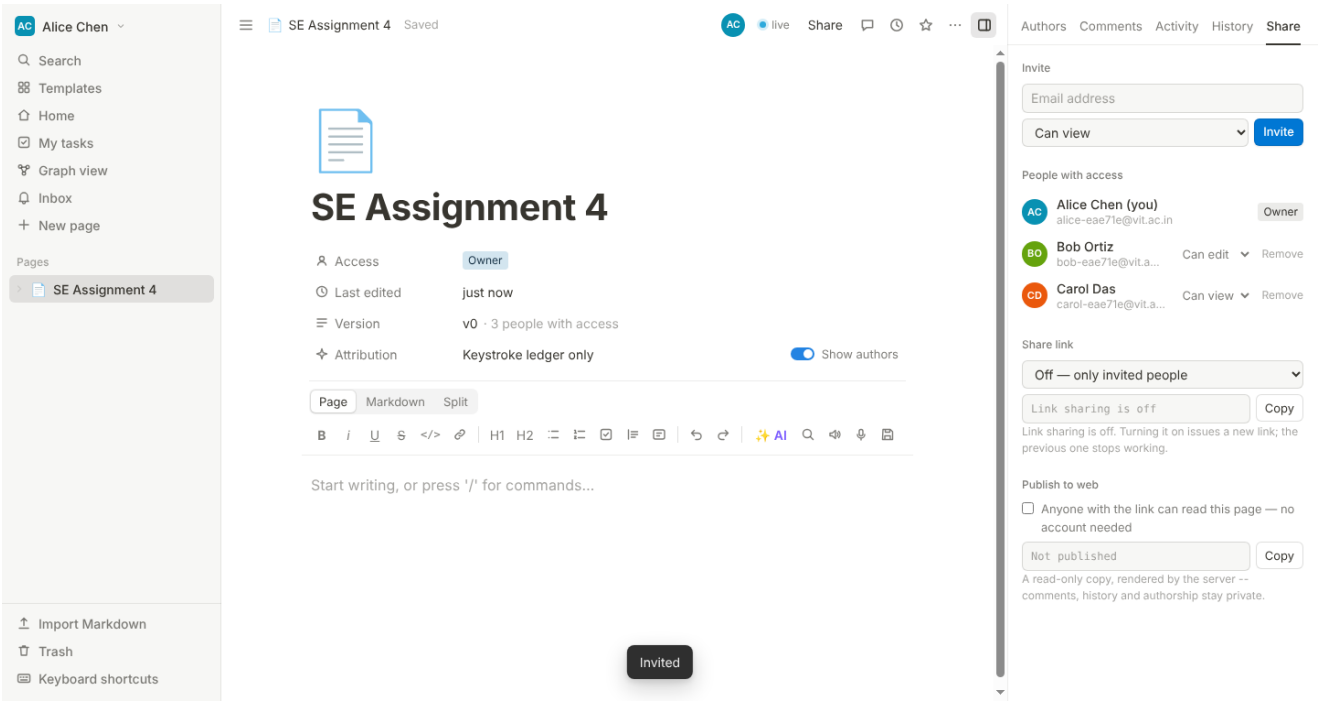

## TC02 - Wrong password is rejected with an error



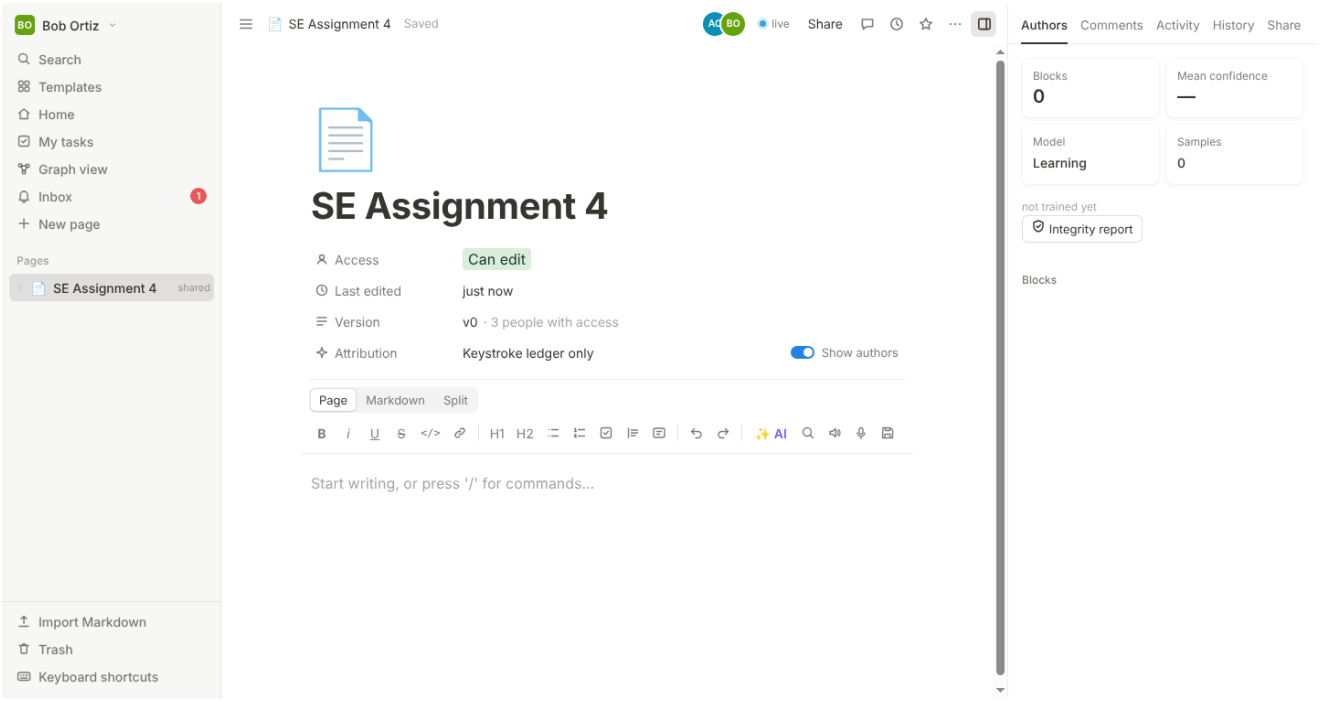
## TC03 - "SE Assignment 4" created and listed in the sidebar



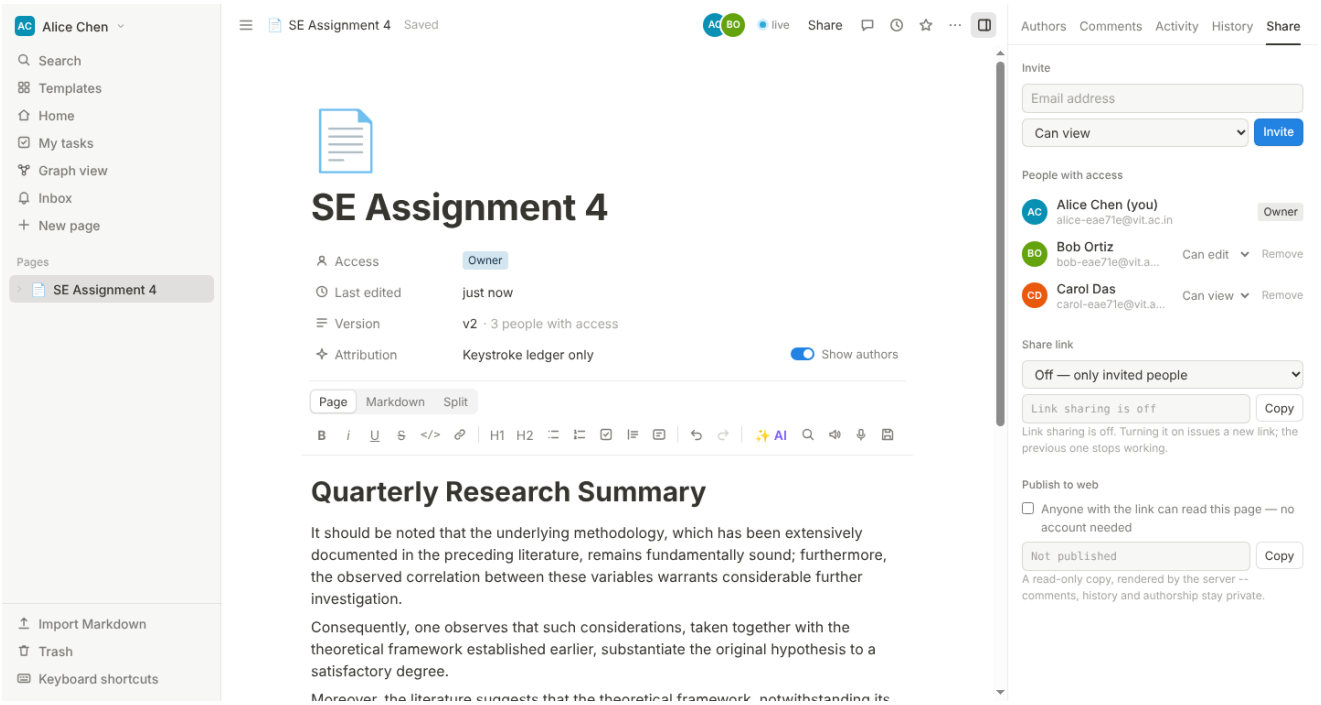

## TC04 - Bob invited as Editor, Carol as Viewer



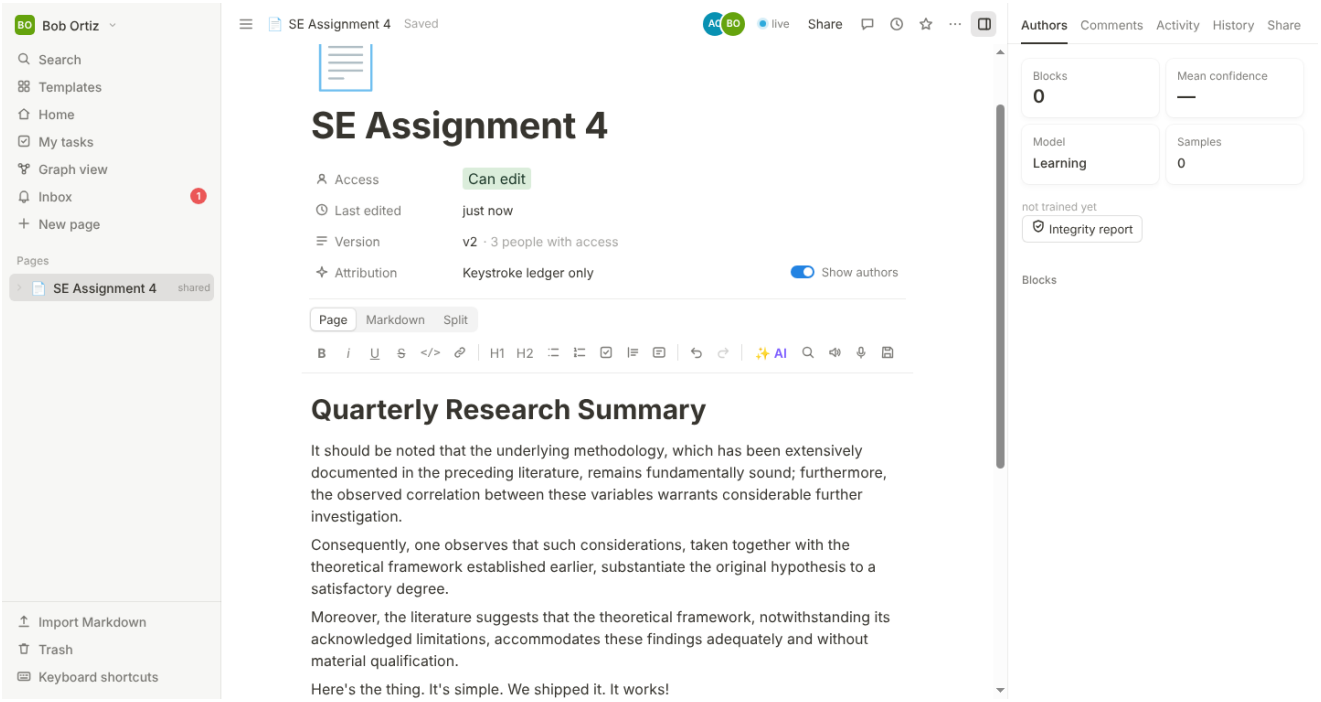
## TC05 - Bob opens the shared page; presence avatars show both users



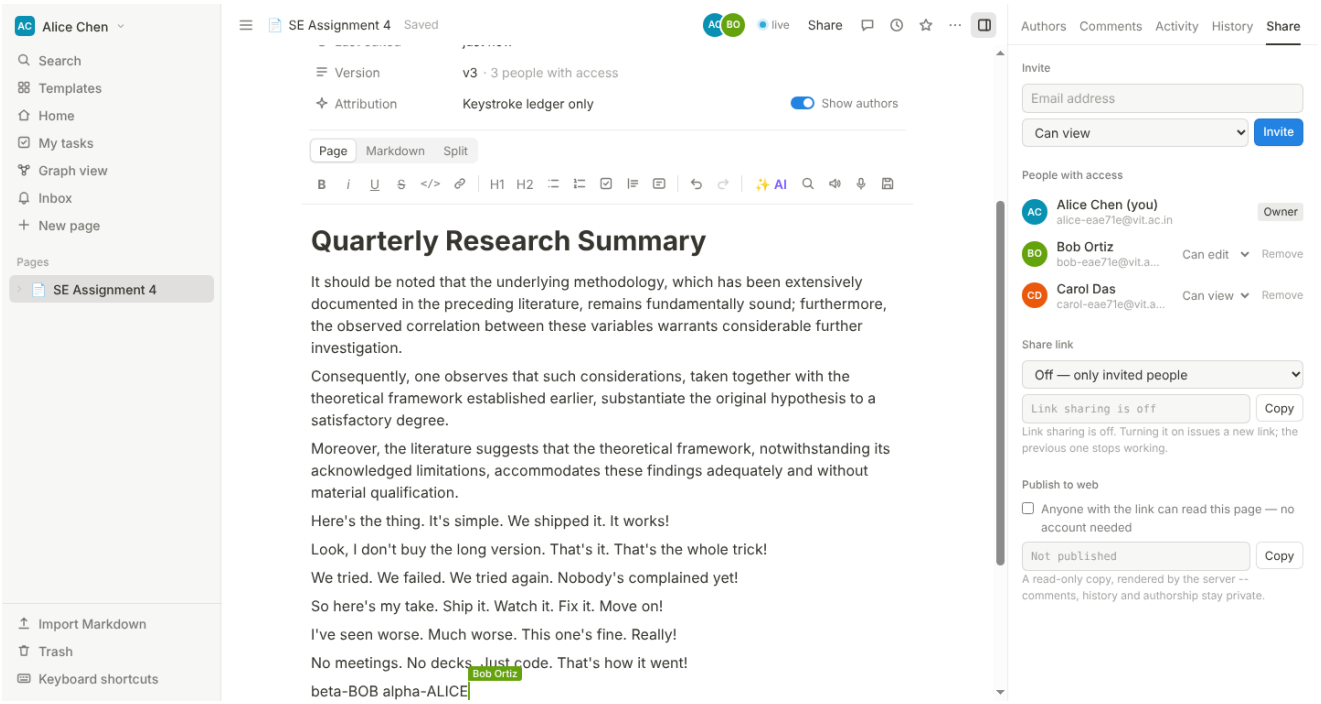

## TC06 - Alice's view after both users typed



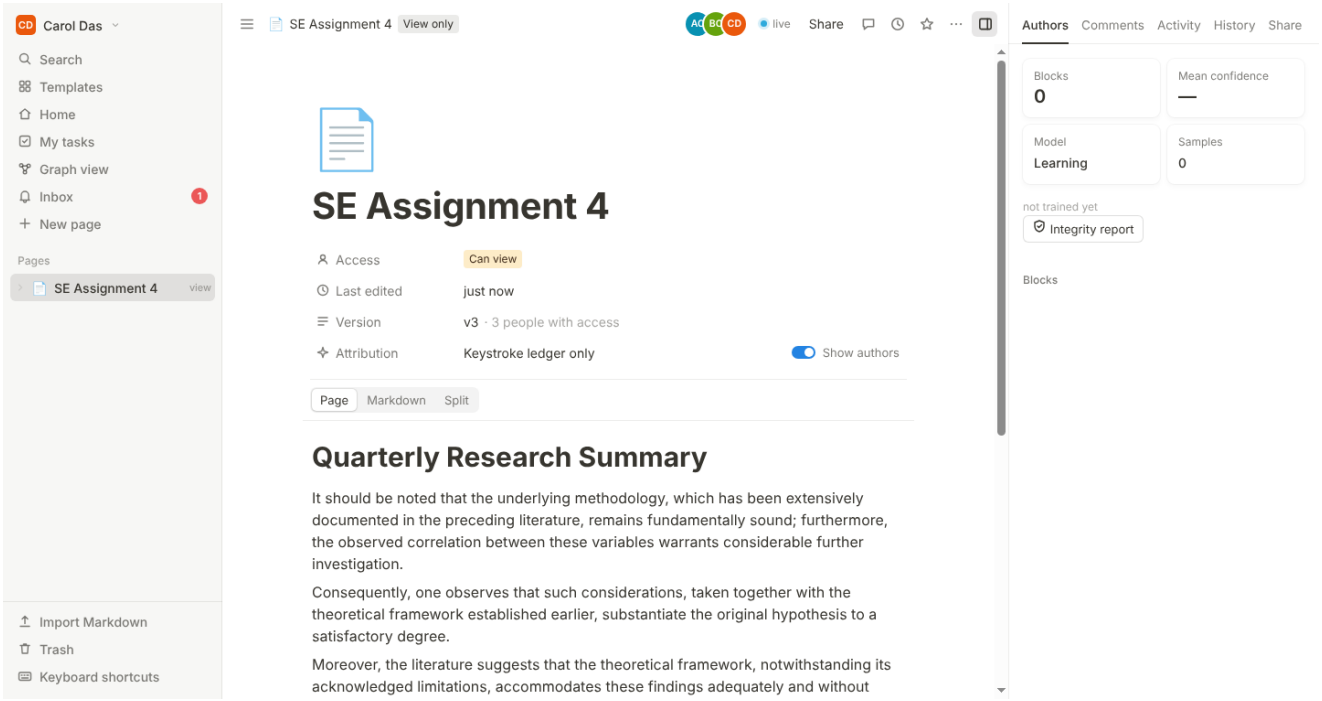
## TC06 - Bob's view: identical text, heading rendered



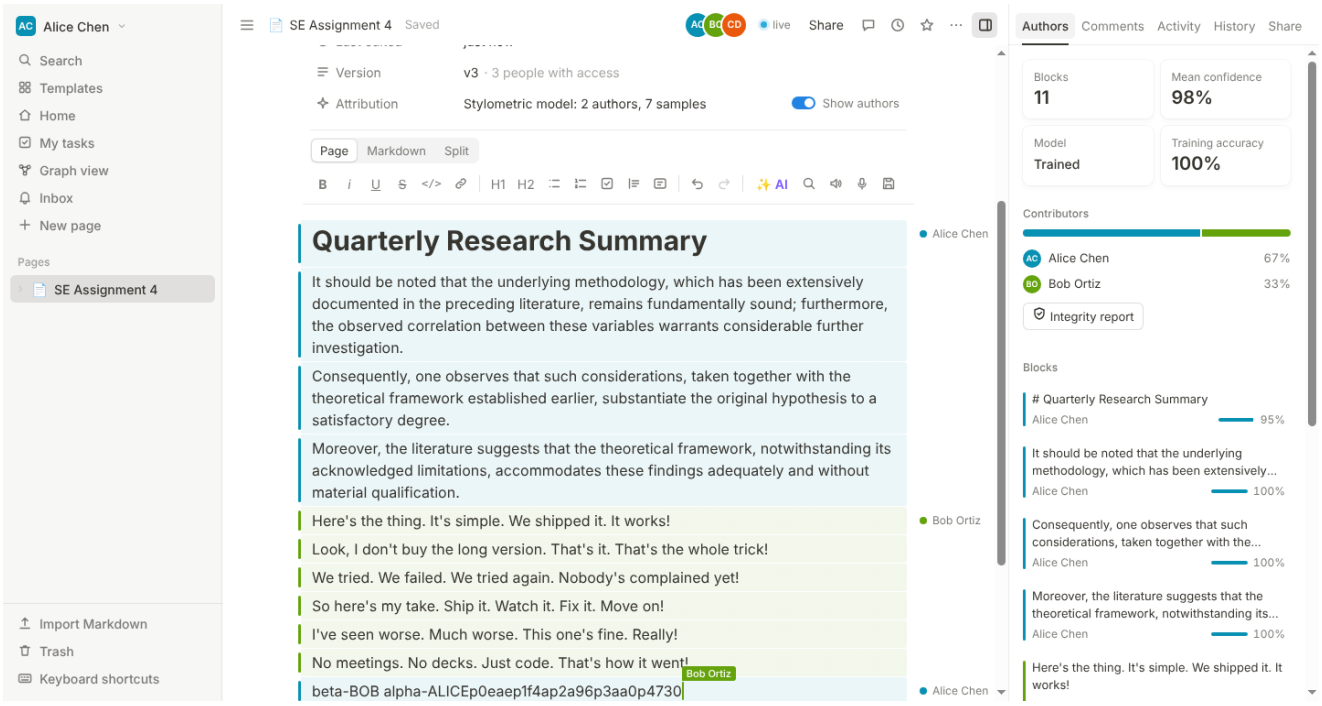

## TC07 - Concurrent edits from both users converged



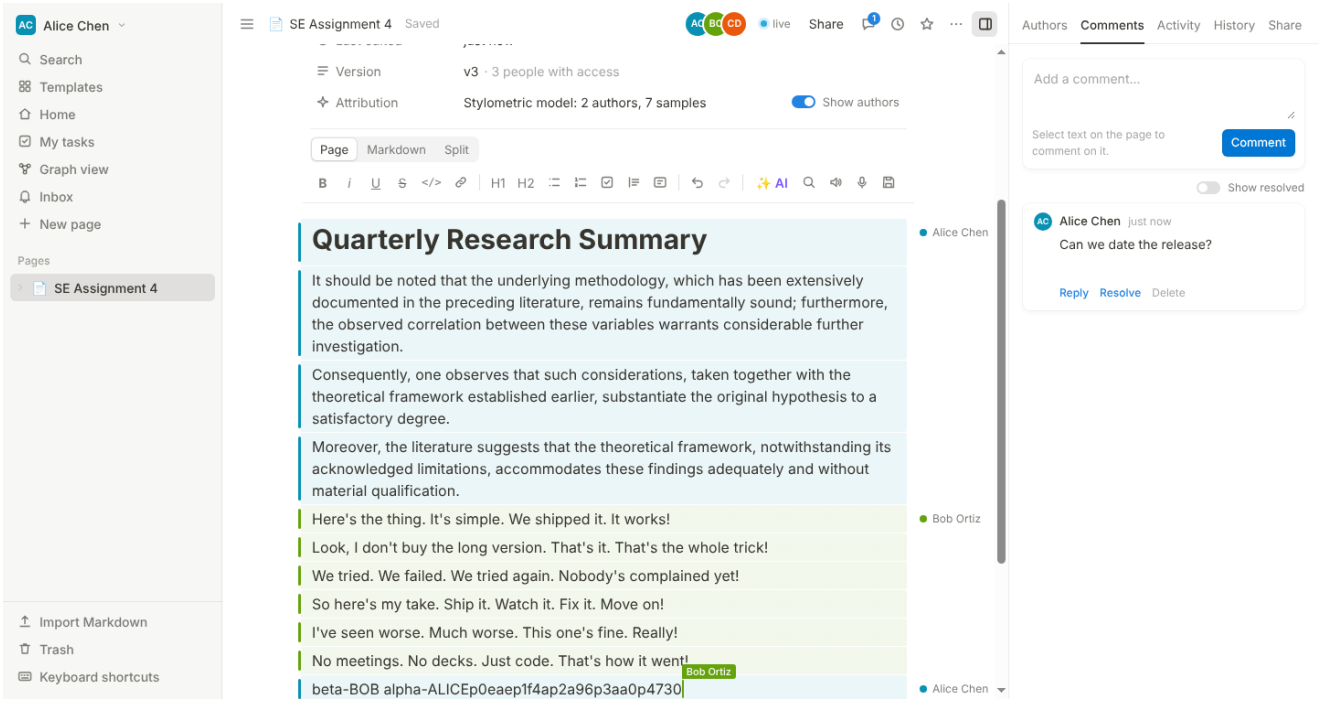
## TC09 - Carol sees "View only"; editor is locked



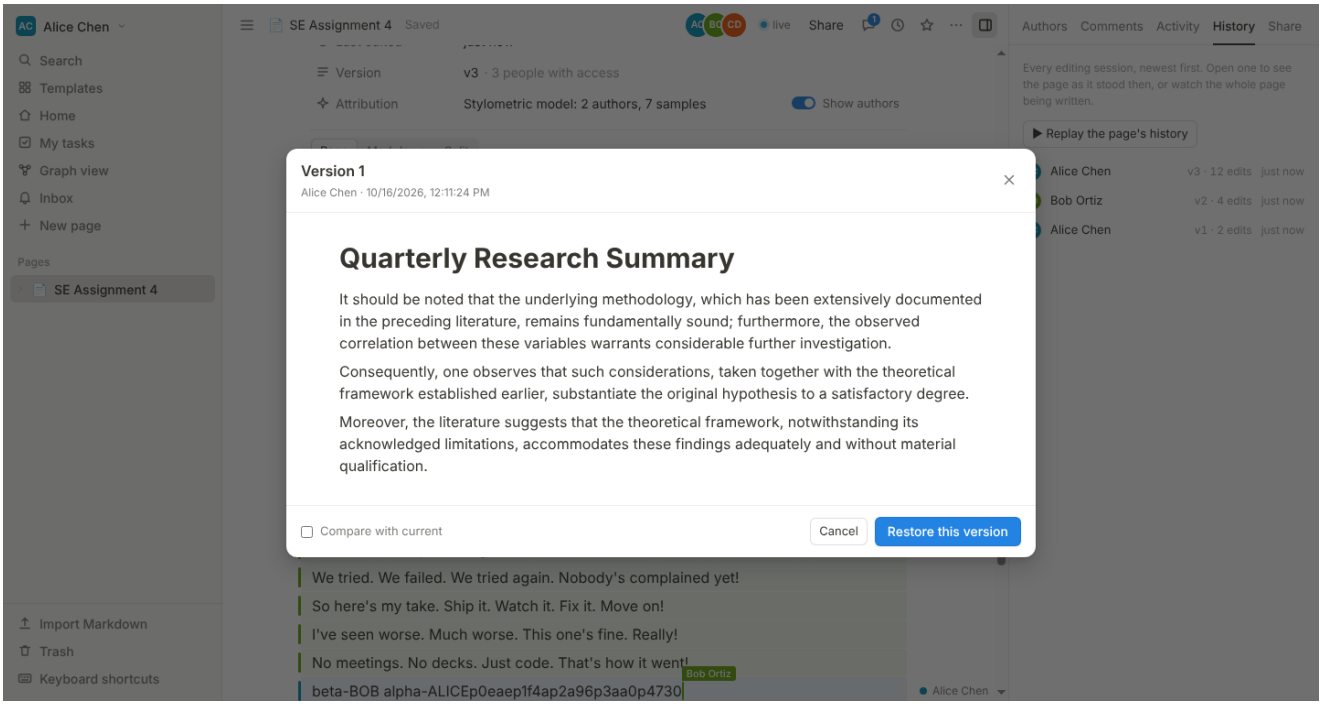

## TC10 - Blocks tinted by author: Alice (blue), Bob (red); model trained



## TC11 - Comment thread created and synced




## TC12 - Version history opened with a saved version



# Test source code (test_collabdoc_e2e.py)

```
"""Selenium end-to-end tests for CollabDoc (Realtime Collaborative Document Server).

Nitish Kumar - 24BCE1520. Runs three real Chrome sessions against the app.
"""
import os, re, threading, time, uuid, statistics
import pytest
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from conftest import BASE_URL, make_driver, HERE

SHOTS = os.path.join(HERE, "screenshots")
os.makedirs(SHOTS, exist_ok=True)
TAG = uuid.uuid4().hex[:6]
PASSWORD = "Selenium@123"
USERS = {
    "alice": ("Alice Chen", f"alice-{TAG}@vit.ac.in"),
    "bob": ("Bob Ortiz", f"bob-{TAG}@vit.ac.in"),
    "carol": ("Carol Das", f"carol-{TAG}@vit.ac.in"),
}
TITLE = "SE Assignment 4"
ALICE_TEXT = (
    "# Quarterly Research Summary\n"
    "It should be noted that the underlying methodology, which has been extensively documented in the "
    "preceding literature, remains fundamentally sound; furthermore, the observed correlation between "
    "these variables warrants considerable further investigation.\n"
    "Consequently, one observes that such considerations, taken together with the theoretical framework "
    "established earlier, substantiate the original hypothesis to a satisfactory degree.\n"
    "Moreover, the literature suggests that the theoretical framework, notwithstanding its acknowledged "
    "limitations, accommodates these findings adequately and without material qualification.\n"
)
BOB_TEXT = (
    "Here's the thing. It's simple. We shipped it. It works!\n"
    "Look, I don't buy the long version. That's it. That's the whole trick!\n"
    "We tried. We failed. We tried again. Nobody's complained yet!\n"
    "So here's my take. Ship it. Watch it. Fix it. Move on!\n"
    "I've seen worse. Much worse. This one's fine. Really!\n"
    "No meetings. No decks. Just code. That's how it went!\n"
)


# ---------- helpers (page-object style) ----------
def wait(d, t=15):
    return WebDriverWait(d, t, poll_frequency=0.05)


def shot(d, name):
    d.save_screenshot(os.path.join(SHOTS, name + ".png"))


def source(d):
    return d.execute_script("return document.getElementById('source').value")


def open_app(d):
    d.get(BASE_URL)
    wait(d).until(lambda x: x.execute_script("return document.readyState") == "complete")
    wait(d).until(EC.element_to_be_clickable((By.ID, "auth-submit")))
    time.sleep(0.3)  # app.js is a module; listeners attach just after load


def register(d, name, email):
    open_app(d)
    d.find_element(By.ID, "auth-toggle").click()
    wait(d).until(EC.text_to_be_present_in_element((By.ID, "auth-submit"), "Create account"))
    d.find_element(By.ID, "auth-name").send_keys(name)
    d.find_element(By.ID, "auth-email").send_keys(email)
    d.find_element(By.ID, "auth-password").send_keys(PASSWORD)
    d.find_element(By.ID, "auth-submit").click()
    wait(d).until(EC.visibility_of_element_located((By.ID, "app")))


def wait_live(d):
    wait(d).until(EC.text_to_be_present_in_element((By.ID, "connection-text"), "live"))


def type_text(d, text):
    ed = d.find_element(By.ID, "preview")
    ed.click()
    ed.send_keys(Keys.CONTROL, Keys.END)
    ed.send_keys(text)


def open_tab(d, tab):
```

```
    d.find_element(By.CSS_SELECTOR, f'#tabs button[data-tab="{tab}"]').click()


# ---------- tests ----------
def test_tc01_signup_reaches_workspace(browsers):
    """TC01 - New users can create an account and land in the workspace."""
    for key, d in browsers.items():
        name, email = USERS[key]
        register(d, name, email)
        assert d.find_element(By.ID, "app").is_displayed()
    shot(browsers["alice"], "tc01_workspace")


def test_tc02_invalid_login_rejected():
    """TC02 - Signing in with a wrong password shows an error and keeps the user out."""
    d = make_driver()
    try:
        open_app(d)
        d.find_element(By.ID, "auth-email").send_keys(USERS["alice"][1])
        d.find_element(By.ID, "auth-password").send_keys("wrong-password")
        d.find_element(By.ID, "auth-submit").click()
        err = wait(d).until(EC.visibility_of_element_located((By.ID, "auth-error")))
        wait(d).until(lambda x: err.text.strip() != "")
        shot(d, "tc02_invalid_login")
        assert not d.find_element(By.ID, "app").is_displayed()
        assert err.text.strip()
    finally:
        d.quit()


def test_tc03_create_document(browsers, state):
    """TC03 - Owner creates a document and renames it; it appears in the sidebar."""
    a = browsers["alice"]
    before = a.current_url
    a.find_element(By.ID, "new-doc").click()
    wait(a).until(lambda x: x.current_url != before and "?doc=" in x.current_url)
    wait_live(a)
    title = a.find_element(By.ID, "doc-title")
    title.clear(); title.send_keys(TITLE, Keys.ENTER)
    wait(a).until(EC.text_to_be_present_in_element((By.ID, "doc-list"), TITLE))
    state["doc_url"] = a.current_url
    shot(a, "tc03_new_document")


def test_tc04_share_editor_and_viewer(browsers):
    """TC04 - Owner invites Bob as Editor and Carol as Viewer (RBAC)."""
    a = browsers["alice"]
    open_tab(a, "share")
    for key, role in (("bob", "EDITOR"), ("carol", "VIEWER")):
        box = a.find_element(By.ID, "invite-email"); box.clear(); box.send_keys(USERS[key][1])
        Select(a.find_element(By.ID, "invite-role")).select_by_value(role)
        a.find_element(By.CSS_SELECTOR, '#invite-form button[type="submit"]').click()
        time.sleep(0.5)
    wait(a).until(lambda x: len(x.find_elements(By.CSS_SELECTOR, "#people-list li")) == 3)
    shot(a, "tc04_shared")


def test_tc05_collaborator_opens_doc_and_presence(browsers, state):
    """TC05 - Bob opens the shared document; both users see two presence avatars."""
    a, b = browsers["alice"], browsers["bob"]
    b.get(state["doc_url"])
    wait_live(b)
    wait(b).until(lambda x: x.find_element(By.ID, "doc-title").get_attribute("value") == TITLE)
    for d in (a, b):
        wait(d).until(lambda x: len(x.find_elements(By.CSS_SELECTOR, "#presence .avatar")) == 2)
    shot(b, "tc05_presence_bob")


def test_tc06_realtime_sync_both_directions(browsers):
    """TC06 - Alice's typing appears for Bob, then Bob's appears for Alice (OT over WebSocket)."""
    a, b = browsers["alice"], browsers["bob"]
    type_text(a, ALICE_TEXT)
    wait(b, 30).until(lambda x: source(x) == ALICE_TEXT)
    type_text(b, BOB_TEXT)
    wait(a, 30).until(lambda x: source(x) == ALICE_TEXT + BOB_TEXT)
    assert source(a) == source(b)
    h1 = b.find_element(By.CSS_SELECTOR, "#preview h1").text
    assert h1 == "Quarterly Research Summary"
    shot(a, "tc06_synced_alice"); shot(b, "tc06_synced_bob")


def test_tc07_concurrent_edits_converge(browsers):
    """TC07 - Alice and Bob type at the same moment; both copies converge to identical text."""
    a, b = browsers["alice"], browsers["bob"]
    base = source(a)
    def go(d, txt):
        ed = d.find_element(By.ID, "preview"); ed.click(); ed.send_keys(Keys.CONTROL, Keys.END); ed.send_keys(txt)
```

```
    t1 = threading.Thread(target=go, args=(a, " alpha-ALICE")); t2 = threading.Thread(target=go, args=(b, " beta-BOB"))
    t1.start(); t2.start(); t1.join(); t2.join()
    wait(a, 20).until(lambda x: "alpha-ALICE" in source(x) and "beta-BOB" in source(x))
    wait(b, 20).until(lambda x: "alpha-ALICE" in source(x) and "beta-BOB" in source(x))
    wait(a, 10).until(lambda x: source(a) == source(b))
    assert source(a) == source(b)
    assert len(source(a)) == len(base) + len(" alpha-ALICE") + len(" beta-BOB")
    shot(a, "tc07_converged")


def test_tc08_sync_latency_under_200ms(browsers, state):
    """TC08 - Keystroke-to-screen latency between two browsers stays under the 200 ms SRS budget."""
    a, b = browsers["alice"], browsers["bob"]
    samples = []
    for i in range(5):
        marker = f"p{i}{uuid.uuid4().hex[:3]}"
        b.execute_script(
            "window.__seen=null;const m=arguments[0];const s=document.getElementById('source');"
            "const t=setInterval(()=>{if(s.value.includes(m)){window.__seen=performance.timeOrigin+performance.now();clearInterval(t
        ed = a.find_element(By.ID, "preview"); ed.click(); ed.send_keys(Keys.CONTROL, Keys.END)
        sent = a.execute_script(
            "const ev=()=>performance.timeOrigin+performance.now();window.__sent=ev();return window.__sent;")
        ed.send_keys(marker)
        seen = wait(b, 10).until(lambda x: x.execute_script("return window.__seen"))
        samples.append(seen - sent)
    state["latency"] = samples
    med = statistics.median(samples)
    print(f"latency samples ms: {[round(s,1) for s in samples]} median={med:.1f}")
    assert med < 200, f"median sync latency {med:.1f} ms"


def test_tc09_viewer_is_read_only(browsers, state):
    """TC09 - Carol (Viewer) can open the document but the editor is not editable."""
    c = browsers["carol"]
    c.get(state["doc_url"])
    wait_live(c)
    wait(c).until(lambda x: "Quarterly Research Summary" in source(x))
    assert c.find_element(By.ID, "preview").get_attribute("contenteditable") == "false"
    assert c.find_element(By.ID, "doc-title").get_attribute("readonly") is not None
    before = source(c)
    try:
        c.find_element(By.ID, "preview").send_keys("HACK")
    except Exception:
        pass
    time.sleep(1)
    assert source(c) == before
    assert "HACK" not in source(browsers["alice"])
    shot(c, "tc09_viewer_readonly")


def test_tc10_ai_authorship_attribution(browsers, state):
    """TC10 - Authors panel tints blocks by author and the stylometric model trains on both writers."""
    a = browsers["alice"]
    open_tab(a, "authors")
    wait(a, 30).until(lambda x: len(x.find_elements(By.CSS_SELECTOR, "#preview .block[data-author]")) >= 5)
    wait(a, 30).until(EC.text_to_be_present_in_element((By.ID, "model-state"), "authors"))
    authors = {e.get_attribute("data-author") for e in a.find_elements(By.CSS_SELECTOR, "#preview .block[data-author]")}
    state["model_state"] = a.find_element(By.ID, "model-state").text
    state["authors_found"] = len(authors)
    shot(a, "tc10_authorship")
    assert len(authors) >= 2, f"expected blocks attributed to 2 authors, got {authors}"


def test_tc11_comment_syncs_live(browsers):
    """TC11 - A comment added by Alice shows up in Bob's comment count without reloading."""
    a, b = browsers["alice"], browsers["bob"]
    open_tab(a, "comments")
    a.find_element(By.ID, "comment-body").send_keys("Can we date the release?")
    a.find_element(By.CSS_SELECTOR, '#comment-form button[type="submit"]').click()
    wait(a).until(lambda x: len(x.find_elements(By.CSS_SELECTOR, "#thread-list .thread")) == 1)
    wait(b).until(EC.text_to_be_present_in_element((By.ID, "comment-count"), "1"))
    shot(a, "tc11_comment")


def test_tc12_version_history(browsers):
    """TC12 - Version history lists saved versions and opens one for preview."""
    a = browsers["alice"]
    open_tab(a, "history")
    wait(a, 20).until(lambda x: len(x.find_elements(By.CSS_SELECTOR, "#version-list li")) >= 1)
    a.find_elements(By.CSS_SELECTOR, "#version-list li")[-1].click()
    wait(a).until(EC.visibility_of_element_located((By.ID, "version-modal")))
    shot(a, "tc12_version_history")
    a.find_element(By.ID, "version-close").click()


def test_tc13_no_javascript_errors(browsers):
    """TC13 - No severe JavaScript errors were logged in any browser during the run."""
```

```
severe = []
for k, d in browsers.items():
    for e in d.get_log("browser"):
        if e["level"] == "SEVERE" and "favicon" not in e["message"]:
            severe.append(f"{k}: {e['message'][:160]}")
assert not severe, "\n".join(severe)
```